

Rakshit Goyal - Alemeno Assignment Submission

Project: Saner

This document is the final submission summary with implementation details, workflow, results, and deliverables.

1. Summary

Saner is an Android React Native marker-scanning app built for the Alemeno internship assignment. It captures a camera snapshot, isolates the marker region, normalizes the output to exactly 300x300, validates the marker using border and anchor structure, and stores accepted results in the app gallery.

- React Native + Expo Android application.
- Native camera preview and snapshot-based capture flow.
- Exact 300x300 processed output.
- Accepts correct markers and rejects incorrect reference markers.
- Stores up to 20 accepted markers in the UI.

2. Deliverables

GitHub Repository: <https://github.com/Rakshitgoyal770/scaner-.git>

Release APK file: Saner-release.apk

APK built locally from: C:\AVRUN\android\app\build\outputs\apk\release\app-release.apk

Detailed approach PDF included separately as: Assignment-Approach.pdf

3. Detailed Explanation

The scanner was designed around structure rather than artwork recognition. The important cues are the marker border, the corner anchor, and a stable normalized crop. Earlier experimentation produced a large single analyzer file, so the final version was refactored into smaller scanner modules to make the implementation easier to read and tune.

- Blob-first crop reduces irrelevant background before scoring.
- Ring-based border detection is more tolerant to rotation than relying only on four rigid strips.
- Corner-anchor analysis distinguishes a valid anchor from oversized or misplaced dark blocks.
- Grayscale previews are generated for debugging and result inspection.
- The output remains exactly 300x300 for assignment compliance.

4. Workflow

- Open the camera preview.
- Align the marker inside the guide frame.
- Tap Capture & Analyze.
- Capture a native snapshot.
- Crop the guide area and tighten around the darkest marker blob.
- Normalize the marker to 300x300.

- Run border and anchor validation.
- If valid, store the processed result in the accepted marker gallery.

5. Results

The following table shows the manual reference baseline used during validation. Each case includes the raw reference image and a grayscale representation, since the detector is intended to be driven mainly by structure instead of color alone.

Case	Reference	Grayscale	Expected
Correct pig marker			Accepted Borders 4/4, Dark corners 1
Correct dog marker			Accepted Borders 4/4, Dark corners 1
Rotated correct marker			Accepted Accepted despite rotation
Incorrect center-square marker			Rejected No anchor corner found
Incorrect oversized-anchor marker			Rejected Anchor square is oversized

6. Achievements

- Created a working Android scanner app with native camera capture.
- Built and tested a release APK for installation on a real device.
- Implemented exact 300x300 processed outputs.
- Refactored the scanner core into smaller, more human-readable files.
- Documented a clear visual baseline with original and grayscale results.

7. Main Files

- app/camera.tsx: camera UI, capture flow, gallery, and metrics.
- app/marker-debug.tsx: sample-image debug browsing screen.
- src/features/scanner/analyzeCapturedMarker.ts: public scanner entry point.
- src/features/scanner/markerAnalyzerCore.ts: blob crop, normalized analysis, scoring.
- src/features/scanner/markerPixelUtils.ts: pixel sampling helpers.
- src/features/scanner/markerPreview.ts: grayscale preview generation.
- src/features/scanner/markerAnalysis.types.ts: shared types and constants.

8. Notes and Limitations

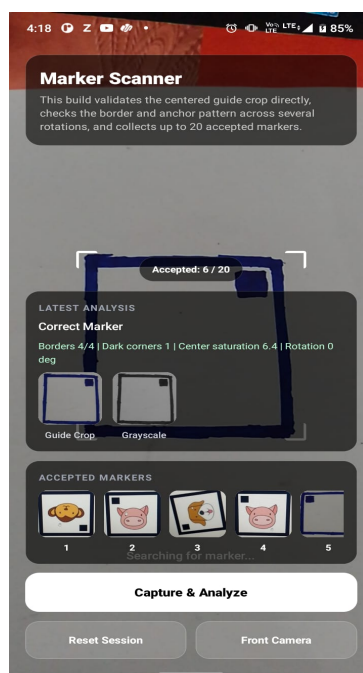
- The current scanner is snapshot-based instead of continuous frame-processor detection.
- The detector is heuristic and tuned for the provided marker family.
- A fuller perspective-warp step could improve extreme-angle robustness further.

9. Final Device Testing Screenshots

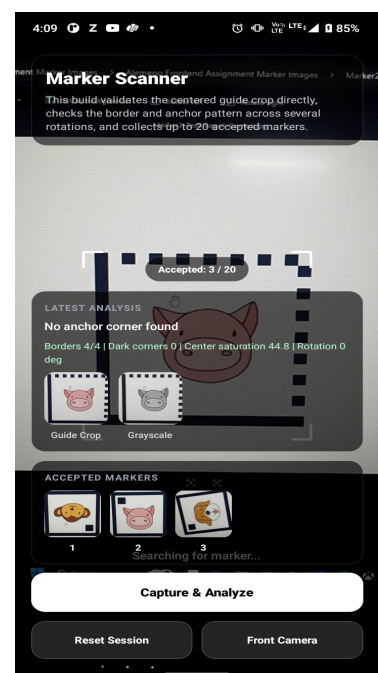
The following screenshots were captured during final on-device testing. They show additional live-screen runs and object-testing scenarios beyond the reference-case summary table above.



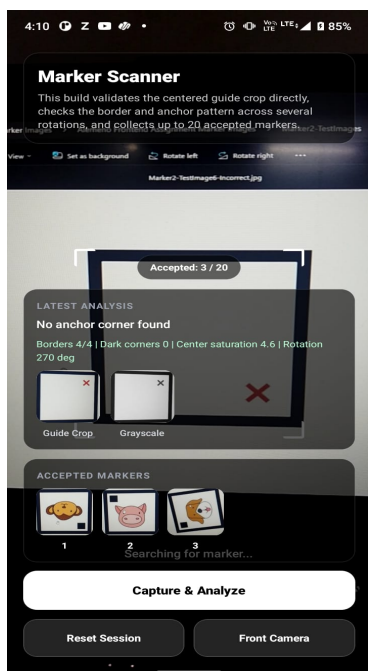
Device test screen 01



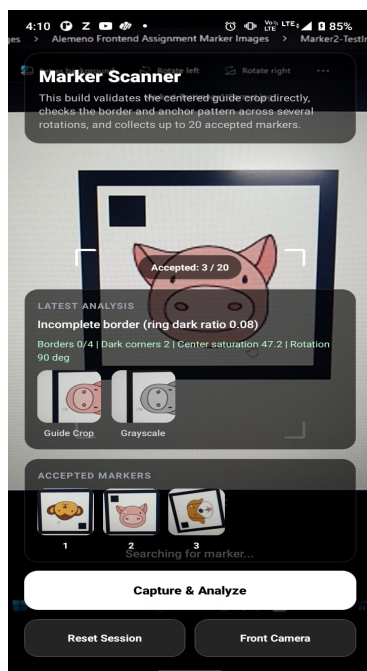
Device test screen 02



Device test screen 03



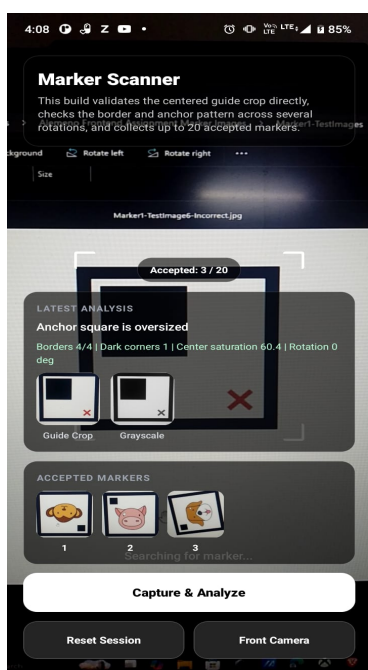
Device test screen 04



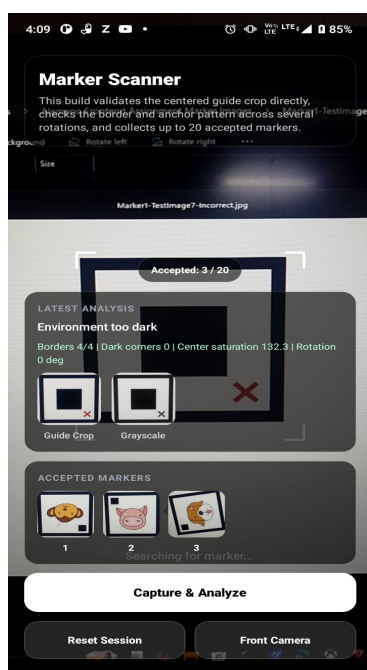
Device test screen 05



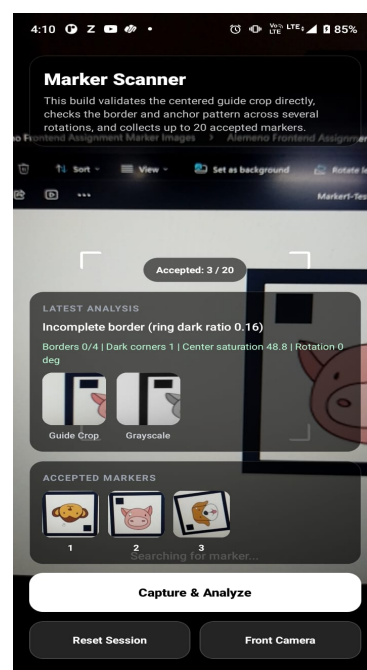
Device test screen 06



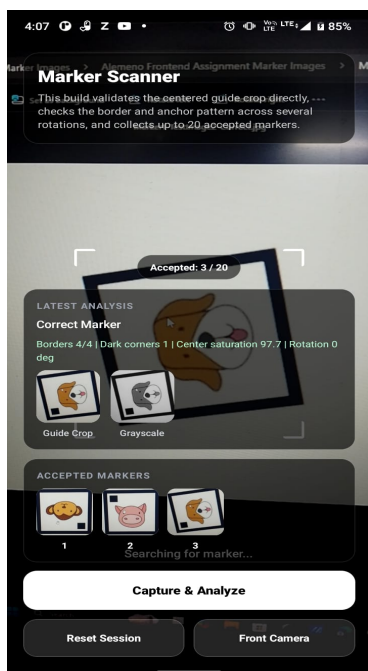
Device test screen 07



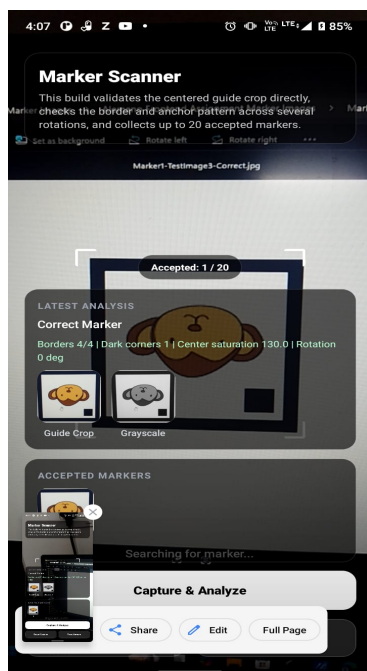
Device test screen 08



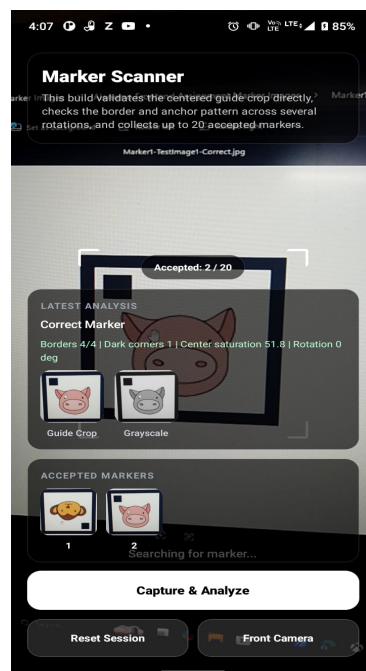
Device test screen 09



Device test screen 10



Device test screen 11



Device test screen 12